RIGA TECHNICAL UNIVERSITY

Faculty of Computer Science, Information Technology and Energy

RTU Liepāja

Nihad Guliyev, Rashad Gasimov, Laurick Spahn-Eigner, Kristen Couty

Academic Bachelor Level Study Program Computer Science

Student ID No 251ADB196, 251ADB248, 260ADB075, 260ADB074

UniConnect

SOFTWARE DEVELOPMENT PROJECT REPORT

Mg. sc. ing

Andrejs Šalms

Liepāja, 29/05/2026

ABSTRACT

EDUCATIONAL TECHNOLOGY, REAL-TIME COMMUNICATION, WEB ARCHITECTURE, ROLE-BASED ACCESS

The accelerated digitalization of higher education has resulted in a blurring of boundaries between personal and academic communication, compelling both students and educators to use poorly-structured, generic platforms for their communication needs. This bachelor thesis proposes the design, development, and deployment of "UniConnect" – a centralized web platform that would cater specifically to university class communication and organization. The research aims to solve the problems associated with the inefficiency of the currently used decentralized solutions by combining features such as real-time group messaging, broadcasting of official announcements, interactive timetables, and a secure polling system. To achieve this goal, a lightweight, containerized solution was created using a Python FastAPI web server with WebSockets and a zero-dependency Vanilla JavaScript frontend. The solution successfully provides a secure and role-based Digital Learning Environment.

The bachelor thesis includes 65 pages, 5 figures, 4 tables, 9 appendices, and 11 references.

ANOTĀCIJA

IZGLĪTĪBAS TEHNOLOĢIJAS, REĀLLAIKA KOMUNIKĀCIJA, TĪMEKĻA ARHITEKTŪRA, LOMĀS BALSTĪTA PIEKĻUVE

Straujā augstākās izglītības digitalizācija ir izpludinājusi robežas starp personīgo un akadēmisko komunikāciju, bieži piespiežot studentus un mācībspēkus paļauties uz fragmentētām, vispārīgām ziņojumapmaiņas platformām, kurām trūkst strukturētu organizatorisko rīku. Šī bakalaura darba mērķis ir projektēt, izstrādāt un ieviest "UniConnect" – specializētu, centralizētu tīmekļa platformu, kas pielāgota tieši universitātes grupu komunikācijai un organizācijai. Pētījums risina pašreizējo decentralizēto metožu neefektivitāti, integrējot reāllaika grupu ziņojumapmaiņu, oficiālu paziņojumu apraidi, interaktīvus sarakstus un drošu balsošanas sistēmu vienotā vidē. Tika ieviests viegls, konteinerizēts risinājums, izmantojot Python FastAPI aizmugursistēmu (backend) ar WebSockets un no atkarībām brīvu Vanilla JavaScript priekšgalsistēmu (frontend). Izveidotā sistēma veiksmīgi nodrošina drošu, lomās balstītu digitālo mācību vidi, kas racionalizē akadēmisko grupu pārvaldību un uzlabo komunikācijas efektivitāti, nepaļaujoties uz trešo pušu komerciālām lietojumprogrammām.

Bakalaura darbs sastāv no 65 lappusēm, tas satur 5 attēlus, 4 tabulas, 9 pielikumus un 11 atsauces uz informācijas avotiem.

TABLE OF CONTENTS

INTRODUCTION

Context and problems

Modern university requires efficient learning management systems and an official mail service to conduct administration and resource distribution. Although these services provide a necessary structure of communication, they can hardly be used as a means of conducting day-to-day communication within certain educational groups like classmates or project teams due to their rigidity and asynchrony.

As a result, people tend to use informal third-party messaging apps such as WhatsApp or Discord to ask questions, schedule meetings or share information instantly. However, it causes some serious problems. Firstly, personal messages are mixed up with academic ones. Secondly, it is hard to track decisions or announcements made during such conversations. Thirdly, there is no clear access control on those platforms, which makes it impossible to understand whether one received a message from a professor or a classmate. We solve this problem by introducing UniConnect, which is a place for these everyday discussions.

Aim of the project

The primary aim of this project is to develop UniConnect, a dedicated web-based coordination and communication platform designed specifically to facilitate daily interactions and group management among students, elected class delegates, and teachers.

Project tasks/objectives

In order to systematically accomplish this goal, several key objectives guide the project's approach. The first step consists of designing a rigid Role

Based Access Control (RBAC) interface that will allow for separation between permissions and views for students, delegates, teachers, and administrators. One of the crucial objectives of the project is creating channels for bidirectional communication in order to replace social media-based conversations with more structured class-specific messaging. Furthermore, the project includes development of tools that help coordinate academic activities, such as interactive timetable, event tracking, and polls for making democratic decisions in class. All these elements will be combined in a lightweight, dependency-free Single Page Application (SPA).

Scope and constraints

The scope of this project is defined to deliver a functional Minimum Viable Product (MVP) focused on real-time group coordination. The MVP includes secure registration, instant messaging, event creation, polling, timetables viewing and priority announcements.

Several features are explicitly out of scope: this platform does not yet aim to replace institutional LMS platforms, meaning grading systems and formal assignment submissions are excluded. Additionally, native mobile applications and built-in video conferencing are excluded from this iteration.

Methods

This development is carried out using an iterative approach inspired by Agile methodology. The development started with the requirement analysis phase where visual prototyping was used to lay down the UI. Detailed Use Cases were then created to convert these concepts into technical requirements.

The development process was done in a series of sprints. The development will end by doing a User Acceptance Testing (UAT) process to ensure that the system works for the intended use cases.

Document structure

The documentation consists of eight major chapters that cover all the aspects of the project's development cycle. Chapters 1 through 4 introduce the theoretical framework, identify the core problem, and describe the objectives and scope of the project. Chapter 5 deals with the functional and non-functional requirements. The system design architecture and technical implementation are described in Chapters 6 and 7. Testing process and its results are discussed in Chapter 8. Conclusion and references follow.

1. BACKGROUND / CONCEPTUAL FRAMEWORK

1.1 Domain overview and context

Today's higher education system relies heavily on the digital infrastructure. All universities have implemented Learning Management Systems (LMS) and university email systems to facilitate formal administration, learning, and grading process. Nevertheless, there exists an additional dimension, which is quite dynamic and requires attention, as well. It refers to daily coordination, for instance, a class of students or a group of students working on a particular project.

For that purpose, students, class delegates, and teaching faculty need fast, informal and easily accessible means to organize classes, exchange schedule information, and resolve any questions immediately. In this case, this sphere was traditionally neglected by institutions, and students used general commercial services to bridge the gap. As a result, today's world of academic cohort management is quite inconsistent, standing somewhere in between the rigidity of institutional tools and the informality of social networks. This aspect is important to consider to create the tool that will unite the two worlds together.

1.2 Key concepts and definitions

To ensure clarity and consistency throughout this documentation, several key conceptual and technical terms are defined below:

- **Role-Based Access Control (RBAC):** A security paradigm where system access and permissions are strictly determined by the user's assigned role.

- **Single Page Application (SPA):** A web application architecture that interacts with the user by dynamically rewriting the current web page with new data from the web server, rather than loading entirely.
- **API/REST:** A standardized architectural style allowing communication between the client application and the server. It relies on stateless HTTP requests to interact with structured data.
- **WebSocket:** A persistent, full-duplex communication protocol over a single TCP connection, used to enable low-latency, real-time functionality.
- **JWT (JSON Web Token):** A secure, compact standard used for safely transmitting information between the client and server as a JSON object, utilized primarily for user authentication and authorization.
- **MVP (Minimum Viable Product):** A development technique in which a new product is built with only the core features sufficient to satisfy early users and validate the fundamental concept.
- **Use Case:** A specific sequence of interactions between a user (actor) and the system designed to achieve a defined, measurable goal.
- **Learning Management Sytem (LMS):** Centralized digital platform designed to create, distribute, and manage educational courses and training programs.

1.3 Overview of existing solutions

The present-day scenario of academic communication is characterized by two different kinds of solutions, but both of them have serious structural drawbacks when it comes to managing cohorts.

On the one hand, there are institutional solutions, such as Moodle and Microsoft Teams. They provide high security and are well-structured, yet their problem lies in the low levels of daily usage for informal purposes.

Moodle forums are asynchronous and might be considered too formal for short questions, while Microsoft Teams is too complicated and resource-intensive.

On the other side are commercial messaging apps like WhatsApp, Telegram, or Discord. They perform exceedingly well on usability and immediacy. Unfortunately, they completely lack the academic context. They do not employ RBAC; therefore, a professor is unable to verify the identity of the student. Moreover, these applications have the inherent disadvantage of mixing personal and academic online existence, causing digital distraction and security issues.

1.4 Requirements perspective

From the analysis of this domain and existing approaches, it becomes clear what a "correct solution" must provide. A system operating in the academic coordination space must blend the real-time responsiveness of commercial messengers with the authoritative structure of an institutional tool.

Typically, systems of this type require clear, distraction-free navigation, real-time data flow for immediate messaging, and highly stable role verification to ensure that official announcements, polls, and so on are trustworthy. Furthermore, usability and performance are paramount, as students expect modern web applications to respond instantly, mirroring the fluid experience of the social platforms they currently use. Building upon these domain expectations, the following chapter (Chapter 2) will precisely define the specific problem UniConnect aims to solve, formally establishing the project's goal and the criteria by which its success will be measured.

2. PROJECT GOAL AND PROBLEM DEFINITION

2.1 Context and stakeholders

Within the domain of higher education management, the project addresses the deficiency in communication for which formal LMS are inadequate. In order to ensure clarity in communication hierarchy, distinct stakeholders are identified. The primary stakeholders are students, who participate in discussion voting, ..., and class delegates, who take active part in managing the cohort. The secondary stakeholders are professors and teaching assistants, who use the application as a direct, noiseless medium for sharing crucial information. Finally, University IT and administration are indirect stakeholders, as they do not use the platform everyday but benefit from its.

2.2 Problem statement

Presently, daily academic coordination depends on messaging applications that do not have any form of structural hierarchy. The result is massive information fragmentation, whereby crucial information issued by either a professor or delegate can be lost amid the noise. Secondly, the lack of proper Role Based Access Control (RBAC) on such commercial platforms leads to ambiguity on the credibility of the information posted. Students end up spending too much time trying to find crucial information. In case this issue remains unsolved, it will continue to result in digital exhaustion, missed academic deadlines, and lack of professionalism in the digital environment.

2.3 Project goal

To provide university students, class delegates, and professors with a dedicated, interactive web-based platform that enables secure real-time messaging, scheduling, and verified polling and more within a role-restricted environment, thereby eliminating information loss and establishing clear boundaries between academic organization and personal social media.

2.4 Success criteria

The success of the project depends of the fulfillment of several criteria that can be measured. First, all of the MVP use cases identified need to work seamlessly without any technical obstacles. Also, there needs to be proven stability, which will involve making sure that the connections between the system and other systems are resilient, that the demo runs without crashing, and that error states are handled effectively using the UI. Thirdly, there should be proper integration of the interactive multimedia features such as the live chat and polling animation. Lastly, the success of the project will be contingent upon the preparation of documentation and verification.

2.5 Target audience and usage scenario

UniConnect is aimed at university students and staff members who need constant, dependable group collaboration. In reality, the software would be used through desktop and laptop computers. The scenarios depend on urgency: For instance, a student could sign in for a brief 30-second session from his phone to read a new announcement or cast his vote in a delegate election. On the other hand, in the course of working on a collaborative assignment, the user can leave the application running on his desktop browser for long to access the live chat. The main constraint in this case is that internet connection needs to be present.

2.6 Assumptions and limitations

The definition of the project goal is based on several early assumptions and technical restrictions. Firstly, it is assumed that the MVP will have its own standalone JWT authentication framework. As far as data storage goes, the architecture will use a local storageopposed to an enterprise cloud solution. From the perspective of functionality, the media capabilities of the platform are limited to text-based messaging; any audio or video conference capabilities are not included. Lastly, the user interface will be designed using only the English language, and although mobile application development may prove valuable in the future, the current scope of the project focuses on desktop functionality only.

2.7 Link to the next chapters

With the problem and project goal strictly defined, the subsequent chapters will operationalize this vision. Chapter 3 translates the core goal into high-level objectives and operational tasks, mapped against a sprint schedule. Chapter 4 establishes the strict scope boundaries, detailing what is explicitly included and excluded. Chapter 5 derives the functional and non-functional requirements necessary to satisfy the use cases, which then directly inform the System Design detailed in Chapter 6.

3. OBJECTIVES AND TASKS

3.1 Objectives

To achieve the primary goal of providing a secure, real-time academic coordination platform, the project is broken down into four high-level objectives:

- **Objective 1 (Design):** Establish and validate the user interface structure through manual prototyping and mutual agreement to ensure a clear, role-based navigation flow.
- **Objective 2 (Core Functionality):** Implement and verify the core Minimum Viable Product (MVP) use cases end-to-end (UC1-7).
- **Objective 3 (Interactivity & Multimedia):** Design and implement interactive user interface components and seamless multimedia elements within the main user flow to enhance user engagement and provide immediate, clear feedback.
- **Objective 4 (Quality & Documentation):** Ensure system stability through rigorous use case testing and finalize the academic documentation.

3.2 Tasks

In order to achieve the mentioned objectives, the following tasks are planned which will produce specific deliverables and indicators of completion.

Task 1 is intended at making manual wireframes for the major dashboards and reaching a consensus regarding the general color scheme and layout of the application. The deliverables include manually designed sketches along with CSS guidelines. Task 1 is completed if all aspects of UI layout

have been prepared to be implemented via HTML/CSS without any further discussion about its structural elements.

Task 2 involves the preparation of the back-end for the FastAPI framework, database and the general project structure which is a necessary step in order to implement UC-01 through UC-07. The deliverable will be a detailed design of the system along with an operational database. Task 2 will be completed if the design of the use case functionality raises no questions anymore, along with the database.

Task 3.1 completion requires the development of the static user interface for the authentication pages aimed at the UI layer of UC-01. The deliverables include static HTML/CSS pages for Landing, Login, and Register screens. Task 3.1 is considered completed once the implemented interfaces fully coincide with the visual prototypes, are responsive on both desktop and mobile views, and contain all necessary fields and buttons.

Task 3.2 implies the implementation of the client-side authentication logic required for the frontend JS layer of UC-01. As a result, the task should produce Vanilla JS scripts that perform form validation and API interaction. Task 3.2 is considered completed once the client successfully validates user inputs, sends a JSON request to the server, securely stores received JWT (e.g., in sessionStorage), and provides the relevant HTTP error code messages ("Invalid password").

Task 3.3 requires the development of the backend authentication API that ensures security of the data transfer. Task 3.3 outputs include the FastAPI endpoints (/login and /register) and their database implementation. Task 3.3 is completed once the passwords are safely hashed before storing, the successful login request receives a signed JWT with the user's role, and the failed login request gets a 401 response.

Task 4.1 focuses on the design and implementation of the chat interface for the UI layer of UC-02. The expected output is the HTML/CSS of the Class Feed including message bubbles, scrolling feed, and input field. Task 4.1 is considered complete when the chat interface looks visually distinct, automatically scrolls down to new messages, and distinguishes between messages of the current user and other participants.

Task 4.2, the implementation of WebSocket client integration on the frontend JS layer of UC-02 is required. The expected output is the Vanilla JS code that will help manage the WebSocket connection. Task 4.2 is complete when the client application establishes the WebSocket connection via JWT, sends messages straight from the input field, and updates the DOM instantly with new messages without reloading the page.

For **Task 4.3**, implementation of the WebSocket server with persistent messages is required for the backend layer of UC-02. The deliverables include the WebSocket route in FastAPI along with the database code responsible for saving the messages. The task will be complete once the server can accept WebSocket connections, authenticate the JWT token, broadcast messages received from clients to other connected clients in the particular class group, and save the messages to database.

Task 5.1 requires creating the announcement creation dialog box and the feed for the UI layer of UC-03. The output includes an HTML/CSS dialog box for creation as well as the announcement feed including different styles for announcements and other messages. The task is complete once the announcement creation dialog box is accessible and the pinned announcements are distinguishable from regular messages with their own styling like a different background color.

Task 5.2 handles the client-side logic for announcements within the frontend JS layer of UC-03. This will produce a Vanilla JS script responsible

for role verification and modal interaction. The task is complete when the "Create Announcement" button is exclusively rendered for users possessing a Delegate or Professor role in their JWT, and when submitting the modal successfully dispatches a correct POST request to the API.

Task 5.3 focuses on developing the role-protected API endpoint for announcements within the backend layer of UC-03. The output is a secure FastAPI POST endpoint (/announcements). The task is finished when the endpoint rigorously verifies that the requester holds Delegate or Professor privileges before saving the record, correctly returning a 403 Forbidden error for standard student accounts.

Task 6.1 entails designing the creation event UI interface and the calendar event list for the UI layer of UC-04. The output of this task will be an HTML/CSS form collecting Date, Time, and Location, together with the event list view. Completion of this task will be done once the interface implements correct data input types such as the use of date pickers and presents the events in an easy-to-read list.

Task 6.2 entails implementing the scheduling client side for the front end JS layer of UC-04. The output will be a vanilla JS script that will process the event form. Completion of this task will be accomplished once the client successfully verifies whether the chosen date is a future date before making the request and then updating the DOM list after a successful API response.

Task 6.3 focuses on the creation of the study event API and its database connection within the backend layer for UC-04. The output will include the required FastAPI endpoints (POST and GET /events). The task will be completed when the server can successfully store the created events in the database and retrieve upcoming events based on the specific user class group.

Task 7.1 focuses on creating the election module. The outputs will include the HTML/CSS voting interface and CSS success animation. Task 7.1 will be considered completed when the interface presents the list of candidates and performs a CSS animation once the success state is reached.

Task 7.2 will focus on implementing the voting interaction and state management for the frontend JS layer of UC-05. The output will include the JavaScript code that manages voting. Task 7.2 will be completed when clicking the vote button sends a POST request, triggers the success animation upon receiving the 200 OK response, and disables the form.

Task 7.3 involves developing the secure voting API equipped with uniqueness constraints for the backend layer of UC-05. The output is a highly secure FastAPI POST endpoint (/vote). Completion is reached when the server safely registers the vote, strictly enforces database constraints to guarantee only one vote per student per election, and proactively returns a 403 error if a double-vote is attempted.

Task 8.1 requires building the administrative panel interface for the UI layer of UC-06. The output will be an HTML/CSS dashboard enabling professors to create groups and manage rosters. The task is considered done when the interface provides a clear, intuitive form for group naming alongside a functional mechanism—such as a list or text area—to easily add student emails.

Task 8.2 focuses on implementing the administrative client logic for the frontend JS layer of UC-06. The deliverable is a Vanilla JS script designed to handle group creation requests. The task is finished when the UI successfully aggregates the group name and student list, formats this data into a JSON payload, properly handles the API response, and dynamically updates the professor's active class list.

Task 8.3 targets the development of the group management API and its corresponding database relations for the backend layer of UC-06. The outputs are the FastAPI POST endpoint (/groups) and the relational database tables linking Users to Groups. The task is complete when the server securely verifies the Professor role, generates the new group record, and successfully links multiple existing user accounts to this newly created group within the database schema.

3.3 Task-to-goal mapping

The following mapping demonstrates the traceability of the project, ensuring that every planned task directly supports a higher-level objective, which in turn delivers the main project goal defined in Chapter 2.
Objective 1 (Design) is delivered by Task 1. Objective 2 (Core Functionality) is delivered by Tasks 2, 3, 4, and 5. Objective 3 (Interactivity & Multimedia) is delivered by Task 6. Objective 4 (Quality & Documentation) is delivered by Task 7.

3.4 Prioritization

In order to make sure that the execution of MVP is realistic given the time limitations dictated by the academic schedule, the development priorities are set through the use of MoSCoW technique. The Must-have priority list includes the core functionality of the platform, namely Task 2 and Task 3 in terms of design, Task 4 in relation to secure user authentication (UC-01), and Task 5 concerning the WebSocket integration (UC-02). The Should-have level includes Task 6 regarding official announcements (UC-03) along with Task 7 relating to the election module (UC-05). As for the Could-have level, it involves complex .ics parsing in terms of official timetable view (UC-07)

as well as a central resource-sharing system allowing professors and students to upload their course materials.

3.5 Milestones aligned with sprints

The development timeline spans four weeks of active coding, organized into a pre-sprint preparation phase followed by four distinct sprints. Because the UI design, system architecture, and baseline API endpoints were fully established before the sprints began, this development phase focused entirely on the technical implementation of the frontend (HTML/CSS and Vanilla JS logic) and the final refinement of backend security, input validation, and data persistence.

Sprint 1, spanning one week, was dedicated to establishing core access by focusing on the complete authentication ecosystem. During this phase, the team implemented the user interface and client-side JavaScript logic for the login and registration flows (UC-01). Simultaneously, necessary backend adjustments were made to ensure secure JSON Web Token (JWT) handling and robust password hashing. The primary milestone for this sprint was successfully enabling a user to securely create an account and gain authenticated access to the platform.

Sprint 2 was also one week long and concentrated entirely on real-time communication, with a particular emphasis on the high-concurrency messaging system. This sprint involved the complex development of both the WebSocket client-side code and the backend broadcast server code (UC-02). The important achievement that came out at the end of Sprint 2 was that users could send each other instant messages in real time in their class groups without having to reload pages.

Sprint 3 was the last sprint that was more than twice long than others and was aimed at integrating the system and testing it thoroughly. The sprint

was related to implementing all other administrative and interactive features, including the Admin Panel used for managing groups (UC-06), the Announcements system (UC-03), and Study Events creation along with the .ics parsing feature (UC-04/07). Also, the sprint included the implementation of the Resource sharing system used for sharing course material and the polling module (UC-05), and CSS animations. Finally, the sprint ended up with the thorough functional testing of all integrated features, resulting in reaching the main milestone – the MVP implementation.

Sprint 4, was dedicated to testing and fixing the bugs we found as well as small changes on the documentation

3.6 Completion measurement

The overall progress and success of this project phase will not be measured by subjective estimates, but by empirical verification. Completion is evaluated by verifying that each Use Case passes its defined acceptance criteria (including handling at least one alternative/error flow per use case). Additionally, the system must demonstrate stability (no crashes during the main WebSocket chat flow), the CSS multimedia animation must render correctly upon voting, and the final documentation must accurately reflect the built system.

4. SCOPE AND CONSTRAINTS

4.1 In-scope

The Minimum Viable Product (MVP) of UniConnect delivers a centralized coordination environment focused on six core functional areas:

- **Role-Based Authentication (UC-01):** A secure authentification system.

- **Real-Time Group Messaging (UC-02):** A live chat interface powered by WebSockets within class groups, allowing instant communication.
- **Official Announcements (UC-03):** A priority broadcasting system allowing Professors and Delegates to inform on critical updates.
- **Study Event & Timetable Management (UC-04/07):** Tools to schedule custom events and view the weekly class schedule (including .ics parsing for official courses).
- **Vote System (UC-05):** A secure polling module with a multimedia success animation to confirm a vote has been cast.
- **Administrator Dashboard (UC-06):** An all in one dashboard for administrators to manage all the users and all the groups.
- **Resource Sharing System:** A centralized repository where users can upload and download course materials.

4.2 Out-of-scope

In order to ensure that the product will be delivered successfully according to the semester schedule, certain non-essential elements have been deliberately excluded from the MVP. First of all, the password recovery functionality has not been included in the current version. In addition, the comprehensive systems for grading and managing examinations were deliberately excluded, as they require dealing with a lot of academic data, which is quite difficult due to the limited time. Moreover, the features related to the submission of homework and assignments were excluded, as these functionalities are fully provided by the institutional LMS. User profile customization was not given high priority either, as the developers would need to focus only on making sure that the communication logic works perfectly.

4.3 Constraints

There were a number of limitations in place for this project, which played a significant role in the final design. First, there was a very tight timeframe, which meant that the system had to be completed and documented during a short period of time, leaving little room for the implementation of secondary functions. In terms of data handling, there are limitations on storage space; all data files and databases are stored locally on the server containing the API, thus avoiding any third-party cloud storage services. Additionally, there are device limitations; the UI is implemented only for desktop browsers. Lastly, technological limitations require the use of Vanilla JS on the front end.

4.4 Assumptions

There are several critical assumptions about the feasibility of the UniConnect projects. It is expected from students and professors to cooperate in good faith by using their official academic email addresses and providing their real names for registration purposes. Therefore, the possibility to delete such non-compliant accounts exists in the MVP version. Also, in terms of client-side integrity, it is assumed that users will use the platform appropriately. Though the backend guarantees security and integrity by strictly enforcing API calls and role-based access checks, the MVP assumes that users will not try to maliciously reverse-engineer, inject, or tamper with the JavaScript frontend. Finally, it is also assumed that the academic community targeted by the project is fluent in English because the interface has been designed in English only.

4.5 Boundaries by dimension

The boundaries that determine the scope of operations in the project include a number of rigid restrictions. First, the scope of functionality is

restricted to four features: communication, scheduling, voting, and resource sharing. In terms of user boundaries, there are three roles established, with no guest access allowed. As far as data boundaries go, all relational data is stored in the database, while all file uploads are stored locally on the server. Lastly, the device boundary allows for the application to be used only on computers.

4.6 Scope change rules

To prevent "scope creep" during the sprints, any requested addition to the functional scope must be documented and justified in the final report. New features are frozen after the start of Sprint 3 to allow for stabilization. Any reduction in the "Must-have" scope requires a clear rationale regarding technical blockers or time exhaustion, which must be reflected in the final testing report.

5. REQUIREMENTS

5.1 Requirements sources and traceability

The requirements that have been laid out in this chapter are based entirely on the project vision stated in Chapter 2, the operational objectives of Chapter 3, and the scope of work delineated in Chapter 4. Every functional requirement is linked to a particular Use Case (UC-xx) and UI screen, thus ensuring that the development process stays on track with the users' requirements.

5.2 Functional requirements

Functional requirements describe the specific behaviors and services the UniConnect system must provide. Please refer to Table 1 for a comprehensive list of these requirements.

5.3 Data requirements

This section specifies the structural models, fields, and constraints of the underlying relational database (managed via SQLAlchemy) required to support the application.

DR-01 (User & Group Management): The User entity holds a unique id, unique email address, encrypted password, and first/last names. The User entity must include the role attribute managed by the Enum (ADMIN, TEACHER, DELEGATE, STUDENT), where the default role is STUDENT. The Group entity represents a class group and holds a unique id, unique name, and an optional field that links to a schedule file (schedule_path).

DR-02 (Real-Time Messaging): For managing message persistence, the Message entity stores the content of the message and the sent_at

timestamp (default value is the current UTC time). To manage chat history, it must have two foreign keys for establishing relationships between messages and users as well as groups.

DR-03 (Official Announcements): For managing announcements with high priority, the Announcement entity holds the title, message content, and the sent_at timestamp. Moreover, the Announcement entity has an urgent boolean flag that will be used to indicate priority styling in UI as well as two foreign keys linking announcements to users and groups.

DR-04 (Events and Timetables): The Event entity describes scheduled events through attributes such as title, description, start/end dates/time, location (latitude/longitude), and foreign key references to its creator and group. An Enum type classifies the event into STUDY, ACTIVITY, EXAM, and COURSE categories.

DR-05 (Polling System): For the sake of data integrity, elections involve three entities that are interrelated. The Poll entity holds the title, is_active flag, and timestamp for creation and expiration of the poll. The Choice entity denotes the candidate running in the election with attributes like name, manifesto, image URL, and a foreign key to its corresponding poll_id. The third entity is Vote, which tracks the act of voting by establishing a relationship between user_id, poll_id, and choice_id.

DR-06 (Resource Sharing System): The Resource object stores essential document metadata, including the title, local server path (file_path), file type, and a category Enum (LECTURE, EXERCISE, STUDENT, OTHER). This

metadata is stored alongside an upload timestamp and foreign keys connecting the file to its uploader and target group.

5.4 Non-functional requirements

Non-functional requirements describe the quality characteristics and technical constraints of the software.

NFR-01 (Reliability): In case of disconnection from the WebSocket server or any other type of error, the system should show an eye-catching red banner notifying the user about the loss of the connection.

NFR-02 (Performance): The time needed to send/receive messages or get informations shall not be more than 500ms in good network conditions.

NFR-03 (Compatibility): The application shall function properly in the latest versions of popular web browsers (Chrome, Edge, Firefox).

NFR-04 (Security): All API routes (except for login/register) shall need a proper JWT token to provide access and perform specific operations.

NFR-05 (Usability): An interactive feedback object shall appear to notify the user of successful completion of an important action, such as voting.

5.5 Requirements prioritization and MVP boundary

The MVP scope is strictly defined by requirements marked as "Must" and "Should" (Quaility improvements for core features). Requirements marked as "Could" are included in the design to demonstrate the platform's potential; however, their full implementation is subject to the successful stabilization of the core features.

5.6 Traceability table

Please refer to Table 2 for a the trabeability table.

6. SYSTEM DESIGN

6.1 Design goals and constraints

The design of UniConnect is driven by the need for a responsive, real-time academic environment. The primary design goals include a desktop-first approach to maximize information density for study coordination and a modular architecture to separate business logic from data persistence. Key constraints from Chapter 4 dictate a "Vanilla" approach for the frontend and a containerized backend using FastAPI to ensure high performance and easy deployment via Docker.

6.2 User interface structure

The application implements a centralized navigation paradigm based on the persistent sidebar component. An unregistered user is redirected to the login or registration page. After logging in, an unregistered user will be redirected to the holding page, while the registered one will get to the main communication center. The sidebar allows switching between functional components while preserving the application-wide state.

Authentication Screens (login.html, register.html): They represent the entrance point of the application. The user's task is to enter his credentials and get the JSON Web Token (JWT), which represents the entrance into the whole application.

Unassigned State (no_groups.html): It represents a temporary holding page for the newly registered users. The only functionality of this page is displaying the waiting message as the user has no access to any of the

groups' modules until he gets assigned to some group by the system administrator.

Communication Hub (chat.html): This page is the default homepage for active users. The feature enables real-time communication, where both students and professors will scroll through their message history and send instant messages to all their class members.

Announcements (announcements.html): The page acts as the exclusive communication zone. All users will have the ability to view the announcements posted on the site, whereas only specific authorized roles (Professors, Delegates) will have the capability to post their notifications.

Polls (polls.html): This is where democratic decisions will be made. The users will make use of this page to check live polls, vote and receive success feedback of multimedia nature.

Timetables (timetables.html): This page is where scheduling takes place. The user will have access to view his/her official calendar and also actively create his/her own study events.

Resources (resources.html): This page is the file repository. The page will help the user download, view and upload files related to his/her courses.

Admin Dashboard (admin.html): This is an interface that is only accessible by administration personnel. This page will enable both Teachers and Administrators to create new class groups and manage student registrations using email lookups.

Please refer to Figure 1 for a detailled use case's flow.

6.3 Core user flows

The system design ensures primary use cases are reachable within two clicks. For UC-02 (Group Messaging), the flow begins with authentication; invalid credentials display an inline error, while successful logins fetch user data and redirect to either a holding page (if unassigned) or the main chat view. Once there, the chat.js script initializes a WebSocket handshake (/ws/chat/{group_id}) to dynamically inject incoming messages into the DOM without page reloads. If this connection fails, a red error banner is immediately displayed at the top of the screen.

6.4 System architecture and module design

UniConnect follows a classic Client-Server architecture with a specific focus on asynchronous communication.

Backend (FastAPI):

- Routers: Modular Python files handle specific REST API endpoints.
- Schemas: Modular Python files handle input validation.
- Core: Group of core components such as the global configuration of the server and the security related features.
- Services: Group of services needed to run the system such as the socket manager or the run of background tasks.
- Database: SQLAlchemy maps Python objects to the database.

Frontend (Vanilla JS):

- Page structure: Individual files (e.g., chat.html) structure each pages.
- Page style: Individual files (e.g., chat.css) styles each pages.
- Page Scripts: Individual files (e.g., chat.js) manage the specific logic and DOM manipulation for each view.

- Navigation side bar: Single .js file for handling page and group switch.
- Common (common.js): Handles shared logic (JWT, error handling, access rights, AppState).

Please refer to Figure 2 for an overview of the system's architecture.

6.5 Data design

The following sub-section provides the definition of the internal data structure that is used by SQLAlchemy ORM and ensures strict relational integrity in all academic components. The persistent data in the application is created using user inputs and stored in a local PostGres database with the use of SQLAlchemy for mapping Python classes to the database tables. Physical files that are uploaded are stored in a local server directory with their respective access paths registered in the database.

Please refer to Figure 3 for an ERD diagram of the database.

6.6 Data flow and state management

UniConnect is built as a Single Page Application (SPA) in which the state is efficiently shared between the client and the FastAPI server. In order to ensure a smooth user experience, a minimal local state is retained by the client. This mainly includes maintaining the current authenticated status using a securely stored JWT, keeping track of the current class group context, and maintaining temporary UI states like open modals, filled form fields, or displayed errors.

All major state changes happen through asynchronous HTTP requests made using the Fetch API, following RESTful architecture strictly. When fetching information through GET requests, the client waits for a response from the server and then dynamically deletes and rebuilds the necessary DOM nodes. If a POST request is successful, the client displays immediate

feedback in the form of closing modals, deleting filled form fields, and applying CSS animations on the required UI elements. In the case of PATCH requests, if the server responds successfully, the client again provides feedback through UI updates. The same happens for DELETE requests where the specific HTML node is immediately deleted from the DOM. Any unsuccessful requests are immediately notified to the user.

Key Flow Example: Casting a Delegate Vote (UC-05)

1. User Action: The student clicks the "Submit Vote" button after selecting a candidate.
2. Client Request: The JS script intercepts the click, prevents default form submission, and sends a POST request to /api/votes with the poll_id, choice_id, and the user's JWT.
3. Server Operation: FastAPI receives the request, extracts the user ID from the JWT, and attempts to insert the Vote object. If the UniqueConstraint fails (user already voted), it returns a 403 Forbidden. If successful, it saves the vote and returns 200 OK.
4. UI Reflection: If 200 Success: The client disables the voting radio buttons (updating the local state), hides the "Submit" button, and triggers the multimedia CSS animation to confirm the action.

6.7 Multimedia design and integration

Multimedia aspects of the app include CSS animations specifically created to offer instant feedback to users and improve their experience with the application. Animations are carefully timed to be shown at certain interactive points where confirmation is needed. Technically speaking, the solution relies on different kinds of visual effects provided by CSS keyframes being used for dynamically generated DOM elements. This way,

there will be absolutely no effect on the performance of page load times. Last but not least, although the animations will not work in older web browsers, this is out of the MVP's scope.

6.8 Design decision justification

The most critical technical decision was the choice of the real-time communication protocol for the chat module.

Decision: WebSockets vs. HTTP Polling for Real-Time Interaction

Criteria	Weight	WebSockets	Long Polling
Real-time	30%	5/5 (Instant)	2/5 (Delayed)
Server resource usage	25%	4/5 (Low overhead)	1/5 (High overhead)
Implementation ease	20%	3/5 (Moderate)	4/5 (Simple)
Scalability	25%	5/5 (High)	2/5 (Low)
Weighted total	100%	4.35	2.15

WebSockets were chosen despite higher implementation complexity because they satisfy the strict performance requirements (NFR-02) and provide the seamless user experience expected in a modern messaging application.

6.9 Traceability summary

The modular architecture outlined in this chapter provides for explicit ownership of all the requirements listed in Chapter 5 by a particular system component, thus ensuring traceability and maintainability due to clear separation of concerns. In the backend part of the application, the core

security and schema modules directly satisfy NFR-04 (JWT-based authorization) and FR-22 (strict validation of user input). At the same time, the backend services employ the singleton socket_manager to facilitate real-time WebSocket broadcasting (FR-04, FR-06), whereas background processes ensure secure lifecycle management of elections (FR-13). The API routers are implemented to match the core MVP use cases (UC-01 to UC-07) using SQLAlchemy as a database implementation covering all necessary structural data constraints (DR-01 to DR-06). As far as the client-side code is concerned, the frontend common module, specifically its AppState singleton and error handling code, satisfy role verification (FR-02), user redirection (FR-03), and feedback about reliability of the system (NFR-01). Finally, navigation scripts provide for seamless group switching without reloading the page (FR-15, FR-21), and page-specific scripts implement the required DOM manipulation for usability features (NFR-05).

7. IMPLEMENTATION

7.1 Implementation environment and tools

UniConnect's development was carried out in Linux and Windows under IDE, supplemented with extensions (Pyright, Prettier...). The back-end is implemented with FastAPI (Python 3.12+), supported by SQLAlchemy for ORM functions and Pydantic for data validation. The back-end service is hosted by Uvicorn and packaged using Docker. In terms of the front-end, the user interface is developed free of frameworks, employing only HTML5, JavaScript, and CSS. For multimedia components, scalable SVG icons and keyframe animation based on CSS are employed. Lastly, the whole system is managed by Docker Compose, providing one command deployment of the whole stack.

7.2 Project structure

The repository follows a strict modular structure, ensuring a clear separation of concerns between the server-side logic and the client-side interface, as illustrated in Figure 4 and Figure 5.

7.3 Implementation of screens/pages

UniConnect frontend uses Vanilla JavaScript, whereby individual screens take care of their DOM manipulations while sharing one AppState. The Login and Register screens act as secure entry points, whereby they handle credentials' inputs and state transitions. In case of a successful authentication, a JSON Web Token (JWT) is stored in the localStorage and the user gets redirected to the Home dashboard. Otherwise, the unsuccessful attempts will automatically display a red error message.

The Chat screen (chat.html) acts as the point of contact for real-time messaging, whereby it contains a scrollable history and dynamic DOM manipulation for new messages. It takes care of edge cases by displaying a "No messages yet" message in case of an empty history or a red warning banner if the connection through WebSocket is lost, meeting the requirements of NFR-01. In addition to that, the Announcements screen (announcements.html) segregates priority and normal announcements into a separate feed. Authorized users are able to open a creation modal that, after submission, executes an asynchronous POST request.

In order to allow for effective democratic voting, the Polls page (polls.html) presents the current election through interactive radio buttons. Once a user successfully casts their vote, the interface changes in order to avoid any repetition. This change includes the disabling of inputs, hiding of the submit button, and launching of the CSS confetti animation. The Timetables page (timetables.html) manages the scheduling of classes by presenting a comprehensive calendar that includes official classes as well as self-created study events, complete with filters and interactive forms.

Lastly, the Resources page (resources.html) serves as the file management system by allowing users to upload and download files using a guided empty state. System administration is completely separate from all other pages and is limited only to the Admin Dashboard (admin.html). This exclusive page allows teachers and administrators to view class groups and enroll students through email search.

7.4 Implementation of core features

UC-01 (Role-Based Authentication): In this use case, we have discussed the role-based authentication mechanism based on OAuth2 and JWT. The client makes a POST request containing credentials at the endpoint

/auth/login. The server will verify the hashed password stored in the database and creates a JWT token. This token will be stored in localStorage and then the client is redirected to the dashboard page. In terms of error handling, in case of an invalid password, 401 Unauthorized is returned resulting in a red-colored banner on the UI. For failed DB connections, 500 Internal Server Error is returned with a generic "Service unavailable" message.

UC-02 (Group Chat in Real Time): This use case defines a real-time messaging service that is based on WebSockets technology. When the chat window loads, chat.js connects to ws://.../chat/{group_id}. Messages that are sent are then emitted from the backend SocketManager, stored in the database, and pushed to all the connections within the group instantly for direct DOM insertion. Alternative flows include silent client-side JS validation, which prevents sending an empty message. In case of network failure, the WebSocket onclose event displays a red alert with "Connection Lost". Currently, one of the limitations is the inability to implement lazy loading.

Use Case UC-03 (Official Announcements): In this use case, there is a priority broadcast system for official announcements. Once the authorized user fills out the form and submits, the client will send a POST request to the /announcements endpoint. The object is saved by the backend, the client will refresh the feed, and the front end will apply unique CSS styling such as red badge for urgent=True items. In terms of error handling, HTML5 required attribute prevents empty title submissions, and API failure will display a general error toast. One downside here is that there are no push notifications.

UC-04/07 (Study Event & Timetable Management): This use case covers timetable management, featuring .ics parsing for official courses and custom event scheduling. Accessing the timetable prompts the client to fetch both parsed .ics data and custom database events to populate the calendar UI. Creating a custom session triggers a POST request to /events, persisting the record and immediately refreshing the UI. For error handling, end times preceding start times trigger a 400 Bad Request and an "Invalid date range" UI alert. If the official .ics file is corrupted, the backend catches the parsing exception and degrades gracefully, showing only custom events alongside a UI warning. Limitations include the lack of recurring events and external calendar synchronization.

UC-05 (Vote System): This use case outlines a secure polling module. Submitting a vote sends a POST request containing the poll_id and choice_id. The backend creates a Vote record; upon a 200 OK response, the UI disables the form and injects .confetti DOM elements for a CSS keyframe animation. If a user maliciously attempts to vote twice via the API, the database's UniqueConstraint("user_id", "poll_id") fails, throwing an IntegrityError and returning a 403 Forbidden to trigger a "You have already voted" message. Server-side timestamp checks similarly reject votes for expired polls. A strict design limitation is complete vote anonymization, allowing administrators to view only total counts.

UC-06 (Administrator Dashboard): This use case defines the administrator dashboard for group management. To enroll a student, a Teacher submits an email address, triggering a POST request to the management endpoint. The backend queries the User table and inserts a

record into the user_groups_association table. For alternative flows, non-existent emails return a 404 Not Found, prompting a "User not found" UI alert. Attempting to add an already enrolled user safely ignores the action and displays a "User already enrolled" badge. A defined MVP limitation is the lack of bulk CSV student uploads.

7.5 Data handling and persistence

For data persistence, the MVP employs a local PostGres database. All important data is immediately persisted; therefore, all chats, polls, and announcements are immediately recorded upon occurrence. In addition, data integrity and security are maintained by validating all incoming payload using Pydantic schemas before inserting into the database. This validation process helps prevent SQL injection and ensures that only correctly formatted data is inserted into the database. Finally, resource management includes handling uploaded files by creating unique filenames on the server to ensure no file is overwritten.

7.6 Multimedia implementation

In order to give instant visual feedback, a success animation is launched when a 200 OK success response is received from the voting API endpoint. This functionality is designed as an extremely optimized and small size CSS animation whereby, after the successful vote, a number of div elements with the .confetti class are appended to the page's HTML DOM dynamically by the client-side code. The simulation of physics behind falling papers is achieved using CSS keyframes through the transform: rotate() and translateY() functions. Performance-wise, by only using Vanilla CSS, there is no possibility of blocking the JavaScript event loop during the animation rendering phase.

7.7 Key implementation decisions and architectural challenges

Going through the process of architectural design was perhaps the most challenging part of implementation, entailing some key technical decisions to be made. Firstly, instead of using the traditional monolithic main.py file, the backend API was divided into separate routers. While it posed some difficulty in terms of design, it greatly simplified debugging and sped up the whole process. As for the frontend, the "no framework" rule had to be applied, but in order not to sacrifice the aesthetics of the modern web, there was no other choice but to use CSS Flexbox and Grid heavily. Thus, a very responsive user interface was obtained, especially in the case of navigation sidebar and real-time chat feed, while the use of bulky UI frameworks like Bootstrap was avoided altogether. Additionally, to ensure consistency of the application state throughout all tabs, the design solution was to introduce an AppState singleton within common.js file.

7.8 Current status

The UniConnect MVP is fully implemented and operational. All "Must" and "Should" requirements from Chapter 5 are met. The "Could" requirement (Resource sharing) is also fully functional, allowing for a complete demonstration of the academic coordination concept.

8. TESTING PLAN AND RESULTS

8.1 Testing objectives and scope

The main purpose of testing in this phase is to ensure that the UniConnect MVP works properly and fulfills all the requirements specified in Chapter 5. Testing will be performed for all major Use Cases (UC-01 to UC-07) as well as for the additional Resource module. Testing will mainly focus on the verification of the "Happy Path" (main success path) and alternative paths (user or system errors). The testing environment is "desktop first," and it will employ the latest versions of Google Chrome, Microsoft Edge, and Mozilla Firefox. The mobile responsiveness will not be tested because of the "desktop first" design principle. Automated load testing, email push notifications, and advanced penetration testing are not included since they exceed MVP boundaries.

8.2 Testing approach and methodology

The testing methodology mainly focuses on manual use case based testing. Manual testing is done against the docker container deployed which is running the FastAPI backend and Vanilla JS front end. The dual testing approach was used where module testing was performed by developers for code developed by others to avoid any confirmation bias. After fixing the defect, whole use case was tested again to check for any new bugs. Seeded database is used which had been set up with 1 Admin, 2 Teachers, 1 Delegate, 3 Students and 2 class groups.

8.3 Test cases derived from use cases

Each implemented Use Case was translated into actionable test cases covering the main success path and at least two alternative flows.

UC-01 (Authentication):

- TC-01A (Happy Path): User enters valid credentials; system issues JWT, saves to localStorage, and redirects to Dashboard.
- TC-01B (Alt - User Mistake): User enters an incorrect password; system prevents login and displays a red error banner (401 Unauthorized).
- TC-01C (Alt - System Issue): User attempts to register with an already existing email; API returns a 400 error, UI prompts "Email already in use."

UC-02 (Real-Time Chat):

- TC-02A (Happy Path): User sends a message; message is broadcasted via WebSocket and appears instantly on all connected clients.
- TC-02B (Alt - User Mistake): User attempts to send an empty message; client-side JS blocks the emission.
- TC-02C (Alt - System Issue): Server connection drops; UI immediately displays a red "Connection Lost" banner (NFR-01).

UC-03 (Official Announcements):

- TC-03A (Happy Path): Professor submits a new announcement flagged as "urgent"; the API saves the record, and the UI immediately updates the feed, applying the specific priority CSS styling (e.g., red badge) to the new item.
- TC-03B (Alt - User Mistake): Professor attempts to publish an announcement with an empty title; HTML5/client-side validation intercepts the action and prevents the form submission, highlighting the required field.

- TC-03C (Alt - System Issue): The user's JWT token expires just before clicking submit; the API returns a 401 Unauthorized, and the UI redirects the user to the login page with a "Session expired" message.

UC-04 / UC-07 (Study Event & Timetable Management):

- TC-04A (Happy Path): Student creates a new custom study event with valid start and end dates; the backend saves the Event object, and the frontend dynamically renders it on the calendar view with the correct color-coding based on the EventType Enum.
- TC-04B (Alt - User Mistake): Student sets the event's "end date" to a time before the "start date"; the system (Pydantic schema validation) rejects the payload with a 422 Unprocessable Entity, and the UI displays "End date must be after start date."
- TC-04C (Alt - System/Data Issue): The official .ics schedule file for the group is missing or corrupted on the server; the backend catches the file parsing error safely, and the UI renders an empty calendar with a graceful "Official schedule currently unavailable" warning instead of crashing.

UC-05 (Vote System):

- TC-05A (Happy Path): Student selects a candidate and submits; UI disables buttons and triggers CSS confetti animation (NFR-05).
- TC-05B (Alt - User Mistake): User attempts to submit a vote without selecting a candidate; HTML5 validation prevents submission.
- TC-05C (Alt - System Issue): User attempts to bypass UI and double-vote via API; database UniqueConstraint rejects the entry, API returns 403 Forbidden, UI shows "Already voted."

UC-06 (Admin Panel):

- TC-06A (Happy Path): Teacher adds a student to a group using their email; student appears in the group roster.
- TC-06B (Alt - User Mistake): Teacher inputs a non-existent email; API returns 404, UI displays "User not found."

Resource Sharing System:

- TC-RES-A (Happy Path): User selects a valid PDF document and clicks upload; the backend securely saves the file to the local directory, records the path in the database, and the UI updates the list to show the new downloadable resource.
- TC-RES-B (Alt - User Mistake): User clicks the "Upload" button without attaching any file; the frontend validation catches the empty input and displays an "Please select a file to upload" alert.
- TC-RES-C (Alt - System/Data Issue): User attempts to download a file that has been accidentally deleted from the physical server disk (but the database record still exists); the FastAPI router returns a 404 Not Found, and the client UI catches this to display a "File not found or moved" error toast.

8.4 Test execution results

The final regression test execution at the end of Sprint 3 is summarized in Table 2 in the Appendix.

8.5 Defects and fixes

Several defects were identified during the testing cycles and later fixed in order to ensure the stability of the MVP.

Defect 01, marked as a major one due to its blocking nature in terms of demoing, happened when the user changed the group by using the sidebar. In such a case, the chat history of the previous group was shown on the screen and messages were sent to the wrong WebSocket room. The reason behind it was the fact that the Vanilla JavaScript DOM failed to remove the previous elements along with the fact that an older WebSocket was still open. To fix the problem, a global clean up function was added to common.js (AppState singleton).

Defect 02 was a very serious one, in which there was a race condition that allowed users to circumvent the limitation of submitting only one vote through fast clicking on the "Submit Vote" button. This defect occurred because of the fact that the function that disables the button on the client side takes time to execute, thus allowing many API calls before the first call is returned. The implemented solution involved enforcing a SQLAlchemy constraint called UniqueConstraint("user_id", "poll_id"). Therefore, even if there are multiple simultaneous calls from the client side, they are rejected by the database through IntegrityError.

Defect 03 was a small defect that involved uploading a resource with the same filename as an already-existing file and unintentionally overwriting the existing file from the server's local hard drive. The cause of this problem was that there was no logic that would generate a unique name for files when uploaded, in the backend router resource.py. This was fixed by making changes to the backend logic such that all files had a UUID or timestamp added to their names before being saved to the hard drive. In addition to these major defects, several minor defects were also fixed.

8.6 Non-functional checks

The interface is user-friendly. Significant destructive or decisive actions (such as voting) provide immediate visual feedback (Confetti animation, button state changes). Empty states ("No resources shared yet") have been appropriately applied to all modules. On typical networks, REST API calls (GET, POST) return within 200 milliseconds. WebSocket messaging is practically instantaneous, comfortably adhering to the less than 500 milliseconds requirement. The CSS Grid and Flexbox layouts of the application work flawlessly in Chrome, Edge, and Firefox without any need for vendor prefixes. Requests to any secured router path (/api/groups) without a proper Bearer token always result in a 401 status.

8.7 Limitations and risks

Although the MVP effectively achieves all its purposes, some technical limitations are still evident. For instance, the chat currently loads the full message history (or a predefined number). This can be a problem as the history of a chat grows enormous in size within a semester. Therefore, lazy loading or pagination is necessary before releasing this version into production. Currently, the tokens are stored in localStorage for easy integration into Vanilla JS code. Although this is tolerable in an MVP version, it poses a risk to XSS attacks.

RESULTS AND CONCLUSIONS

Goal restatement

Designing and development of Minimum Viable Product (MVP) of UniConnect was the main objective of this project. UniConnect is a unified platform for academic coordination which will work in real time to ensure easy communication, scheduling, and democratized decision making in university classes. It aims to solve the problem of fragmented academic tools by providing a unified platform.

Task-by-task completion summary

The project was carried out by tackling each operational task described in Chapter 3. First of all, the system architecture and database design was successfully implemented. A solid base was created using FastAPI and SQLAlchemy to build a reliable backend, whereas the frontend part of the application was developed with HTML, JavaScript and CSS. Overall, all the required use cases have been successfully implemented (real-time messaging, polls, announcements, and timetables). Lastly, the whole system was containerized with Docker, and comprehensive use-case testing was performed to confirm successful completion of both success and alternative paths. Throughout the project lifecycle, the team successfully completed the planned tasks, adapting to challenges as necessary. The majority of the core objectives were met within time. Please refer to Table 4 for detailed breakdown of individual contributions.

Results summary

The final delivered system is a full-fledged web application capable of performing all MVP use cases. Authentication using JWT is possible for

users (UC-01); they can communicate in real-time (UC-02), broadcast important messages (UC-03), schedule events (UC-04/07), conduct secure voting (UC-05), and manage the roster of groups (UC-06). In addition to this, the Resource Sharing System, which was a "could-have" initially, was also implemented to upload and download files. All communications take place in a persistent relational PostGres database.

Testing outcome summary

Results of system verification demonstrated high reliability, with all MVP scenarios successfully verifying through acceptance tests with 100% completion rate. The most important bug that has been fixed is WebSocket de-synchronization while changing groups, which was solved by introducing socket cleanup logic to clients.

Development of UniConnect was a great way for us to gain experience in developing full-stack apps. Backend technologies utilized turned out to be highly efficient, as our modular FastAPI routes, Pydantic models, and database schemas guaranteed smooth and flawless performance. Contrary to our expectations, developing SPA application using only Vanilla JS was quite difficult because of DOM-manipulation challenges.

Future work

If the project were to be developed beyond the MVP phase, the following logical next steps would be pursued:

1. Chat Pagination: Implementing lazy-loading for the chat interface to maintain performance as message histories grow over a semester.
2. Institutional SSO: Integrating the authentication system with existing university infrastructure (e.g., LDAP, OAuth2 via Microsoft/Google) for seamless student onboarding.

3. Push Notifications: Adding Web Push API or email integrations to alert users of urgent announcements even when the application is closed.
4. Progressive Web App (PWA): Enhancing the frontend to serve as a PWA, granting users a native-like mobile experience with offline capabilities.

LIST OF REFERENCES

Docker Inc. *Docker Documentation: Docker*. Online. 2024. Available from: https://docs.docker.com/. [viewed 2026-05-22].

Fette, I., Melnikov, A. *The WebSocket Protocol*. Online. Internet Engineering Task Force (IETF), 2011. RFC 6455. Available from: https://datatracker.ietf.org/doc/html/rfc6455. [viewed 2026-05-22].

FRAZER. *UniConnect Source Code*. Online. GitHub, 2026. Available from: https://github.com/pixilie/uniconnect. [viewed 2026-05-27].

FRAZER. *UniConnect Web Application*. Online. 2026. Available from: https://uniconnect.pixilie.net/. [viewed 2026-05-27].

Google Fonts. *Material Symbols & Icons*. Online. 2024. Available from: https://fonts.google.com/icons. [viewed 2026-05-22].

Google. *Material Design 3*. Online. 2024. Available from: https://m3.material.io/. [viewed 2026-05-22].

Mozilla Developer Network (MDN). *CSS Layout: Grid and Flexbox*. Online. 2024. Available from: https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout. [viewed 2026-05-22].

Refactoring UI. *Heroicons*. Online. 2024. Available from: https://heroicons.com/. [viewed 2026-05-22].

SQLAlchemy Authors. *SQLAlchemy 2.0 Documentation: The Database Toolkit for Python*. Online. 2024. Available from: https://docs.sqlalchemy.org/. [viewed 2026-05-22].

Tiangolo. *FastAPI framework, high performance, easy to learn, fast to code, ready for production*. Program. 2024. Available from: https://fastapi.tiangolo.com/. [viewed 2026-05-22].

APPENDICES

ID	Requirement Description	Priority	Related UC
FR-01	The system shall allow users to authenticate using an institutional email and password.	Must	UC-01
FR-02	The system shall verify the user's role and group during the login process.	Must	UC-01
FR-03	The system should redirect the user to a waiting page if no group is assign to him.	Should	UC-01
FR-04	The system shall establish a persistent WebSocket connection for real-time group messaging.	Must	UC-02
FR-05	The system shall display basic message's related metada.	Must	UC-02
FR-06	The system shall broadcast sent messages to all connected members of a group instantly.	Must	UC-02
FR-07	The system shall allow professors and delegates to publish announcements with a title and body.	Must	UC-03
FR-08	The system shall display the newest official announcements first.	Should	UC-03
FR-09	The system shall allow the user to create normal priority and urgent announcement	Should	UC-03
FR-10	The system shall allow users to create events (title, date, and location).	Must	UC-04
FR-11	The system shall allow the professors and delegates to create multiple choices votes.	Must	UC-05
FR-12	The system shall record a single, anonymous vote per student for a given vote.	Should	UC-05

FR-13	The system should show the vote results in some way at the end of the poll.	Must	UC-05
FR-14	The system shall allow administrators to create class groups, assign students via email and manage everything through a single dashboard.	Must	UC-06
FR-15	The system shall give the possibility to administrator to switch beetwin group easily.	Should	UC-06
FR-16	The system shall allow user to review their class schedule and custom events.	Must	UC-07
FR-17	The system shall display different type of events in different colors.	Should	UC-07
FR-18	The system shall give to user the possibility to view more details about the event by clicking on it.	Should	UC-07
FR-19	The system shall allow users to upload course resources to a group.	Could	
FR-20	The system shall list shared resources and allow their download by all group members.	Could	
FR-21	The system shall allow user to switch beetwin groups easily without page reload.	Should	Global
FR-22	The system shall validate every user input to prevent invalid data being proccessed.	Should	Global

Table 1: Detailed functional requirements for the UniConnect system

UC	FR	Related screens	Verification method
UC-01	01, 02, 03	Login screen, Waiting page	Manual test of valid/invalid credentials and routing for users with no assigned group.
UC-02	04, 05, 06	Chat tab	Real-time broadcast test using two parallel client sessions.
UC-03	07, 08, 09	Announcement tab	Visual confirmation of chronological sorting and UI styling for urgent priority announcements.
UC-04	10	Event creation form	Form submission test and check if displayed in schedule.
UC-05	11, 12, 13	Polls tab	Attempting a double-vote to trigger restriction; verifying annoucement after poll expiration.
UC-06	FR-14, 15	Admin dashboard	Confirm action execution through a student/teacher user.
UC-07	16, 17, 18	Schedule view	Visual test verifying color-coding logic and ensuring the details modal opens on click.
Global	19, 20, 21, 22	Resources tab, General UI	File upload/download test; manual triggering of empty/invalid inputs to verify system rejection.

Table 2: Traceability table

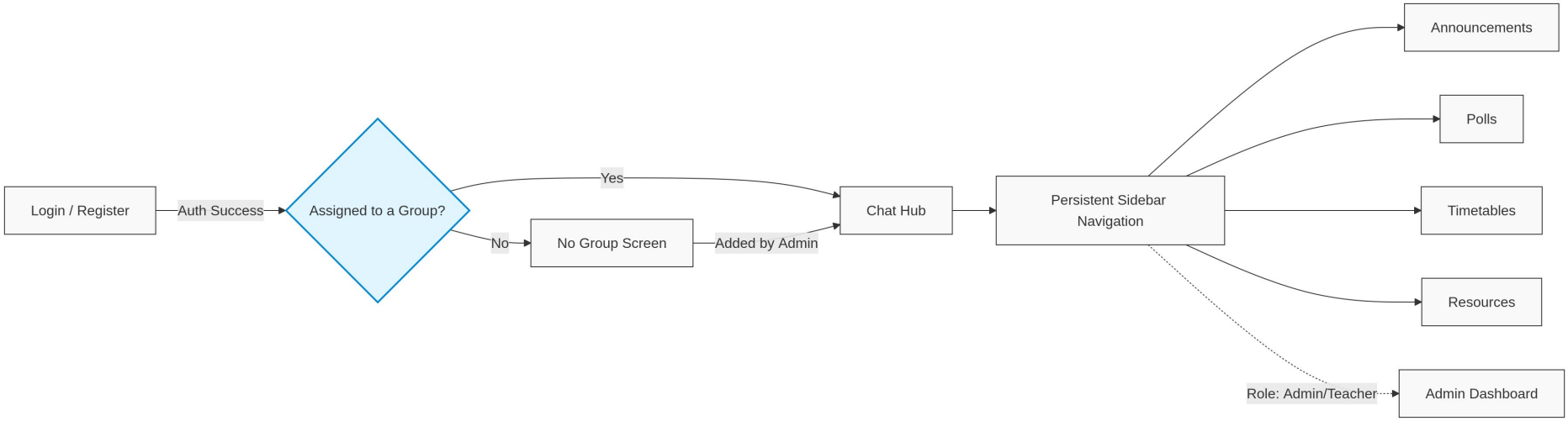

Fig. 1. Application screen map and navigation flow

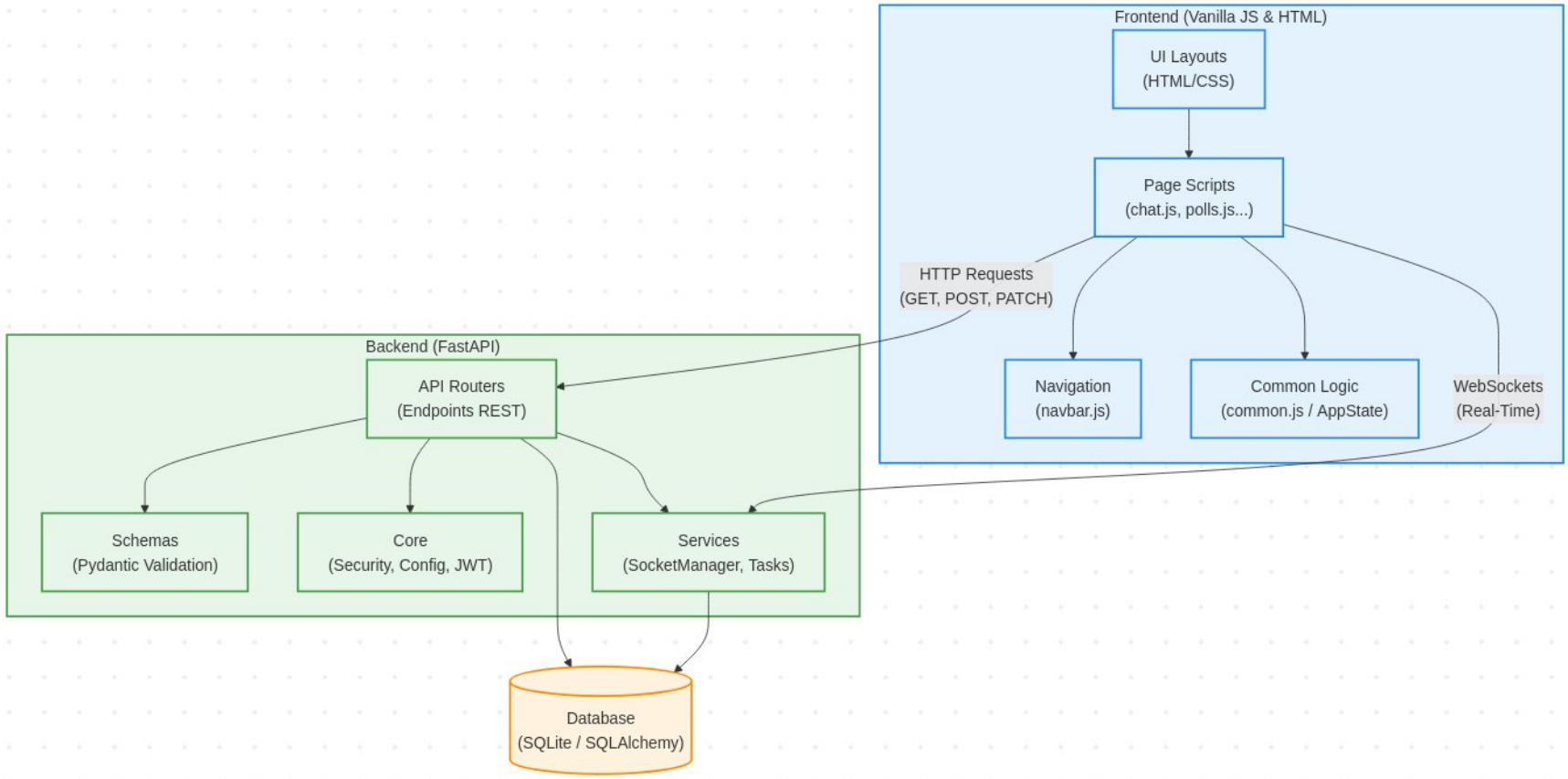

Fig. 2. UniConnect System Architecture and Component Dependencies

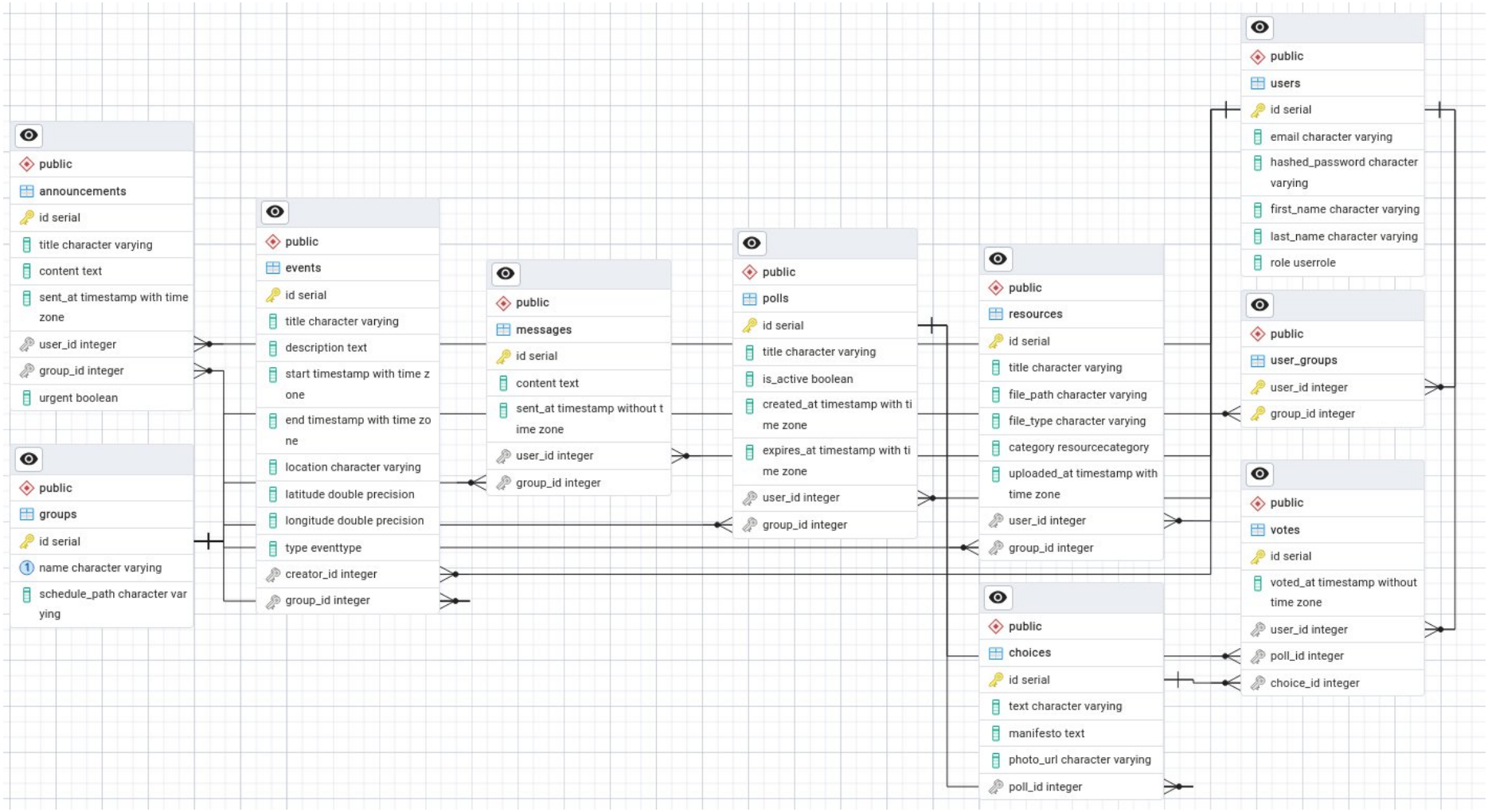

Fig. 3. ERD of UniConnect database

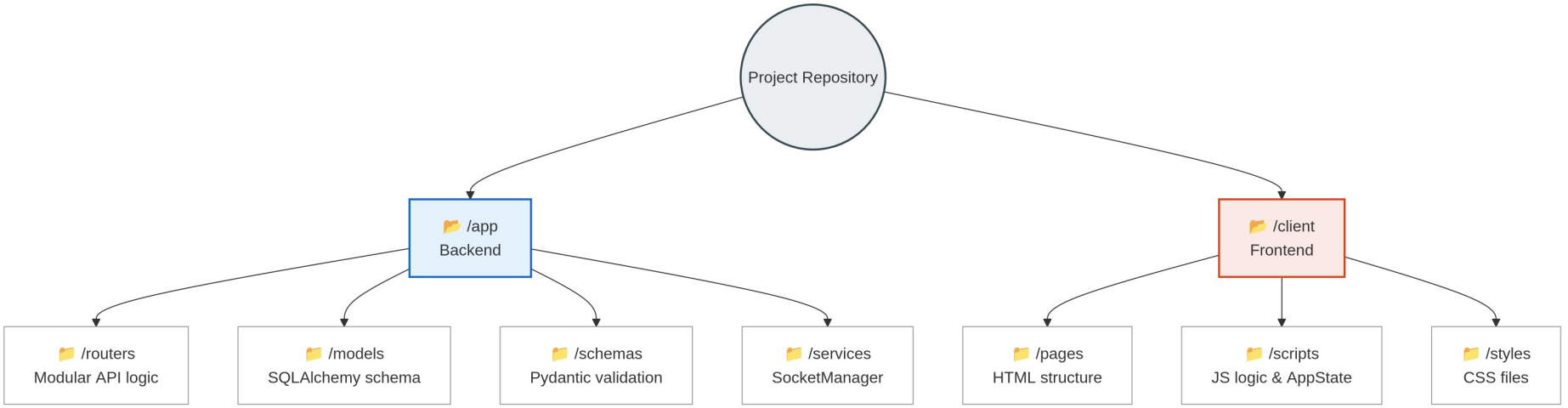

Fig.4. Project structure

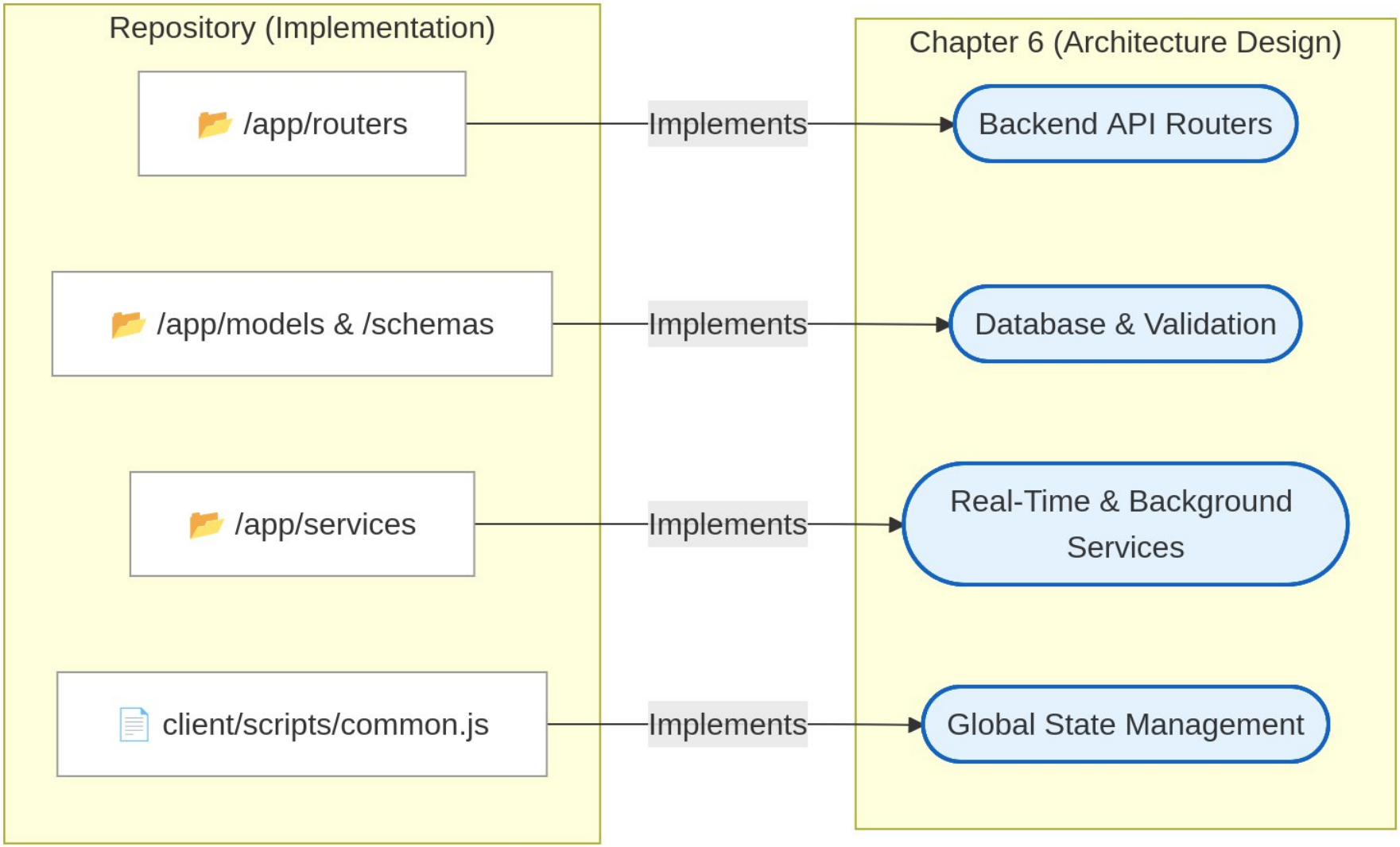

Fig. 5. Traceability mapping between the physical repository and Chapter 6 architectural modules

UC	TC	Scenario	Requirement	Result	Notes / Fix Reference
01	01A	Login success	FR-01	Pass	Valid JWT correctly saved to storage.
01	01B	Invalid credentials	FR-02, FR-22	Pass	Handled natively by FastAPI OAuth2 logic.
01	01C	Duplicate email registration	FR-22	Pass	API returns 400 error; UI alerts user.
02	02A	Message broadcast	FR-04, FR-06	Pass	WebSocket latency verified under 100ms.
02	02B	Empty message prevention	FR-22	Pass	Client-side JS and backend block emission successfully.
02	02C	WS disconnect	NFR-01	Pass	Fixed by adding onclose JS event trigger.
03	03A	Create urgent announcement	FR-07, FR-09	Pass	Priority CSS badge applies correctly on feed refresh.
03	03B	Empty title validation	FR-22	Pass	HTML5 validation effectively blocks submission.
03	03C	Expired JWT token	NFR-04	Pass	UI catches 401 Unauthorized and redirects to login.
04 07	04A	Create custom event	FR-10, FR-16	Pass	Event renders correctly on the Timetable view.
04	04B	Invalid date range	FR-22	Pass	Backend Schema rejects payload with 422 error.

07					
04 07	04C	Missing .ics file	NFR-01	Pass	Backend parses safely; UI renders graceful warning.
05	05A	Cast vote	FR-12, NFR-05	Pass	Confetti animation triggers without UI lag.
05	TC-05B	Empty vote submission	FR-22	Pass	HTML5 validation prevents unselected submission.
05	05C	Prevent double vote	FR-12	Pass	DB UniqueConstraint successfully rejects duplicate.
06	06A	Add user to group	FR-14	Pass	Association table updates correctly in database.
06	06B	Add unknown user	FR-14	Pass	UI correctly catches 404 Not Found error.
RES	RESA	Upload PDF document	FR-19, FR-20	Pass	Files save to local disk with unique identifiers.
RES	RES-B	Empty file upload	FR-22	Pass	Frontend successfully blocks empty attachment.
RES	RES-C	Handle missing file	NFR-01	Pass	Graceful failure if file is missing from server disk.

Table 3: Summary of final regression test execution results for sprint 3

Task description	Laurick	Nihad	Rashad	Kristen
Instant messaging system				
User interface (HTML, CSS)	10%	45%	45%	0%
Client side logic (JS)	80%	5%	5%	10%
WS manager & API endpoints & Server logic (Python)	5%	0%	0%	95%
Announcement tab				
User interface (HTML, CSS)	0%	50%	50%	0%
Client side logic (JS)	75%	10%	10%	5%
API endpoints & Server logic (Python)	0%	0%	0%	100%
Resource sharing				
User interface (HTML, CSS)	10%	45%	45%	0%
Client side logic (JS)	75%	10%	10%	5%
API endpoints & Server logic (Python)	10%	0%	0%	90%
Scheduling system				
User interface (HTML, CSS)	20%	40%	40%	0%
Client side logic (JS)	80%	5%	5%	10%

API endpoints & Server logic (Python)	20%	0%	5%	75%
Polling system				
User interface (HTML, CSS)	10%	45%	45%	0%
Client side logic (JS)	100%	0%	0%	0%
API endpoints & Server logic (Python)	0%	5%	5%	90%
Administrator dashboard				
User interface (HTML, CSS)	0%	45%	45%	10%
Client side logic (JS)	70%	10%	10%	10%
API endpoints & Server logic (Python)	10%	10%	10%	70%
General				
Documentation	20%	20%	20%	40%
Designing	25%	25%	25%	25%
Project's architecture conception	20%	20%	20%	40%

Table 4: Summary of individual team member contributions and task completion